Chandan kumar thakur

Day 5 – Arrays Advanced (Two Sum + Majority Element)

1️⃣ Two Sum Problem

Problem Statement:

Given an array arr and a target sum target, find indices of two numbers such that they add up to target.

Example

arr = [2, 7, 11, 15]
target = 9

Output

[0, 1] // 2 + 7 = 9

Approach 1 – Brute Force

Steps:

Do nested loops

Check if arr[i] + arr[j] == target

Print indices

Java Code – Brute Force

```
public class TwoSum {

    public static void main(String[] args) {

        int[] arr = {2,7,11,15};
        int target = 9;

        for(int i=0; i<arr.length; i++){
            for(int j=i+1; j<arr.length; j++){
                if(arr[i] + arr[j] == target){
                    System.out.println("Indices: " + i + ", " + j);
                }
            }
        }

    }

}
```

Output

Indices: 0, 1

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

2️⃣ Majority Element

Problem Statement:
Find the element that appears more than n/2 times in an array of size n.

Example

arr = [2,2,1,1,2,2,3]

Output

Majority element = 2

Approach 1 – Brute Force

Count frequency of each element

Check if count > n/2

Time Complexity: $O(n^2)$
Space Complexity: $O(1)$

Brute Force Approach Step by Step

Loop 1 – Element select karna

Array me har element ko candidate ke roop me select karte hain.

Loop 2 – Count occurrences

Inner loop me array traverse karke check karte hain ki current element kitni baar array me aaya.

Check condition

Agar count > n/2 → wahi element majority hai.

Chandan kumar thakur

Java Code Example – Brute Force

```
public class MajorityElementBruteForce {

    public static void main(String[] args) {

        int[] arr = {2,2,1,1,2,2,3};
        int n = arr.length;

        for(int i = 0; i < n; i++){
            int count = 0;

            // Count occurrences of arr[i]
            for(int j = 0; j < n; j++){
                if(arr[j] == arr[i]){
                    count++;
                }
            }

            // Check if count > n/2
            if(count > n/2){
                System.out.println("Majority element = " + arr[i]);
                break;
            }
        }

    }

}
```

Step by Step Execution

Array: [2,2,1,1,2,2,3]

i = 0 → arr[0] = 2

Inner loop counts: 2 appears 4 times

Check: 4 > 7/2 → true

Print → Majority element = 2

Break

i = 1 → arr[1] = 2 → Already found, code breaks

i = 2 → arr[2] = 1 → Count = 2 → 2 > 3.5? ❌

Similarly, baaki elements check hote.

Time Complexity

Outer loop = n times

Inner loop = n times

Total = n * n = $O(n^2)$

Space Complexity

Sirf ek counter variable use ho raha hai → O(1)